# 성능 최적화 기법

## 개요

파일코인 proof 시스템은 대용량 데이터 처리와 복잡한 암호학적 연산을 수행하기 때문에 성능 최적화가 매우 중요합니다. 이 문서에서는 파일코인 구현체에서 사용되는 주요 성능 최적화 기법들을 상세히 분석합니다.

## GPU 가속을 이용한 머클 트리 구축 최적화

### GPU 가속 지원 구조

파일코인 proof 시스템은 CUDA와 OpenCL 두 가지 GPU 프레임워크를 지원합니다:

```
// Cargo.toml 특성 설정

cuda = [

"storage-proofs-core/cuda",

"storage-proofs-porep/cuda",

"storage-proofs-post/cuda",

"bellperson/cuda",

"filecoin-hashers/cuda",

]



opencl = [

"storage-proofs-core/opencl",
```

```
"storage-proofs-porep/opencl",
"storage-proofs-post/opencl",
"bellperson/opencl",
"filecoin-hashers/opencl",
]
```

## GPU 트리 빌더 활성화 조건

GPU 트리 빌더는 특정 조건에서만 활성화됩니다:

```
// settings.rs에서 GPU 트리 빌더 활성화 조건
pub fn use_gpu_tree_builder<Tree: MerkleTreeTrait>(&self) -> bool {
        self.use_gpu_tree_builder && TypeId::of::<Tree::Hasher>() == TypeId::of:
}

pub fn use_gpu_column_builder<Tree: MerkleTreeTrait>(&self) -> bool {
        self.use_gpu_tree_builder && TypeId::of::<Tree::Hasher>() == TypeId::of:
}
```

**핵심 특징:**

- **해시 함수 제한**: GPU 가속은 Poseidon 해시 함수에서만 지원
- **배치 처리**: `max_gpu_tree_batch_size` 설정으로 GPU 배치 크기 조절 (기본값: 700,000)
- **컬럼 빌더**: `max_gpu_column_batch_size` 설정으로 컬럼 배치 크기 조절 (기본값: 400,000)

### GPU 메모리 관리 최적화

GPU 가속 시 메모리 사용량을 최적화하기 위한 전략:

1. **배치 크기 조절**

- 대용량 트리 구축 시 메모리 부족 방지
- GPU 메모리 용량에 따른 동적 배치 크기 조절

2. **GPU 락 메커니즘**

- 여러 프로세스가 동시에 GPU를 사용할 때 직렬화
- 메모리 경합 방지 및 안정성 확보

3. **폴백 메커니즘**

- GPU 사용 불가 시 자동으로 CPU 처리로 전환
- 시스템 안정성 보장

## 멀티코어 SDR(Stacked DRG Replication) 처리 방식

### 멀티코어 SDR 아키텍처

```
// settings.rs의 멀티코어 SDR 설정
pub struct Settings {
    pub use_multicore_sdr: bool,
    pub multicore_sdr_producers: usize, // 기본값: 3
    pub multicore_sdr_producer_stride: u64, // 기본값: 128
    pub multicore_sdr_lookahead: usize, // 기본값: 800
}
```

## 코어 그룹 관리 시스템

파일코인은 하드웨어 토폴로지를 분석하여 최적의 코어 할당을 수행합니다:

```
// cores.rs의 코어 그룹 생성 로직
fn create_core_units(
core_count: usize,
group_count: usize,
cores_per_unit: usize,
allowed_cores: &hwloc::CpuSet,
) -> Vec<Vec<usize>>
```

**핵심 최적화 전략:**

1. **캐시 지역성 최적화**
   - L3 캐시를 공유하는 코어들을 그룹화
   - 캐시 미스 최소화로 성능 향상
2. **NUMA 인식 스케줄링**
   - 메모리 접근 지연 최소화
   - 코어와 메모리 간 거리 최적화
3. **동적 코어 바인딩**

```
pub fn bind_core(core_index: CoreIndex) -> Result<Cleanup> {
// 특정 코어에 스레드 바인딩
// SMT/하이퍼스레딩 고려한 단일 논리 프로세서 선택
```

```
}
```

### 멀티코어 SDR 처리 파이프라인

1. **Producer 스레드**

- 레이블 생성 작업을 병렬로 수행
- `multicore_sdr_producer_stride` 단위로 작업 분할

2. **Consumer 스레드**

- Producer가 생성한 레이블을 소비
- `multicore_sdr_lookahead` 설정으로 미리 읽기 최적화

3. **메모리 동기화**

- 원자적 연산을 통한 스레드 간 동기화
- 락 프리 알고리즘으로 성능 최적화

## 부모 캐시 시스템의 메모리 최적화

### 부모 캐시 구조

```rust
// cache.rs의 ParentCache 구조
pub struct ParentCache {
    pub path: PathBuf,
    num_cache_entries: u32,
    cache: CacheData,
    pub sector_size: usize,
    pub digest: String,
```

```
}


struct CacheData {
        data: Mmap, // 메모리 맵핑된 데이터
        offset: u32, // 노드 단위 오프셋
        len: u32, // 노드 단위 길이
        file: LockedFile, // 파일 락
}
```

## 메모리 최적화 기법

1. **부분 캐시 로딩**

```
// 전체 캐시를 메모리에 로드하지 않고 필요한 부분만 로드
fn shift(&mut self, new_offset: u32) -> Result<()> {
        let offset = new_offset as usize * DEGREE * NODE_BYTES;
        let len = self.len as usize * DEGREE * NODE_BYTES;
        self.data = unsafe {
                MmapOptions::new()
                .offset(offset as u64)
                .len(len)
                .map(self.file.as_ref())?
        };
```

```
}
```

2. **캐시 크기 조절**

- `sdr_parents_cache_size` 설정 (기본값: 2,048)
- 메모리 사용량과 성능 간 균형 조절

3. **순차 접근 최적화**

```
// 캐시는 순차적으로만 읽어야 함 (ascending order)
ensure!(
        node >= self.cache.offset + self.cache.len,
        "cache must be read in ascending order"
);
```

## 캐시 검증 및 무결성

1. **해시 기반 검증**

```
// SHA256 해시를 통한 캐시 무결성 검증
let mut hasher = Sha256::new();
hasher.update(&data);
let hash = hasher.finalize();
```

2. **프로덕션 매니페스트 검증**

- `parent_cache.json` 매니페스트와 비교
- 무결성 검증 실패 시 자동 재생성

# 디스크 I/O 최적화 및 캐시 전략

## 메모리 맵핑 최적화

```
// memory_handling.rs의 CacheReader 구조
pub struct CacheReader<T> {
file: File,
bufs: UnsafeCell<[Mmap; 2]>, // 더블 버퍼링
size: usize,
degree: usize,
window_size: usize,
cursor: IncrementingCursor, // 원자적 커서
consumer: AtomicU64, // 소비자 위치
}
```

## 더블 버퍼링 전략

1. **윈도우 기반 읽기**

```
// 두 개의 버퍼를 번갈아 사용하여 연속적인 읽기 성능 향상
pub unsafe fn slice_at(&self, pos: usize) -> &[T] {
let window = pos / self.window_element_count();
let targeted_buf = &self.get_bufs()[window % 2];
// ...
```

```
}
```

2. **비동기 버퍼 교체**

- 한 버퍼를 읽는 동안 다른 버퍼를 미리 로드
- I/O 대기 시간 최소화

## 디스크 I/O 최적화 기법

1. **메모리 락킹**

```
fn allocate_layer(sector_size: usize) -> Result<MmapMut> {
match MmapOptions::new()
.len(sector_size)
.map_anon()
.and_then(|mut layer| {
layer.lock()?; // 메모리 페이지 락킹
Ok(layer)
})
}
```

2. **배치 쓰기 최적화**

- `column_write_batch_size` 설정 (기본값: 262,144)
- 작은 쓰기 연산들을 배치로 묶어 성능 향상

3. **파일 락 메커니즘**

```
// parameter_cache.rs의 파일 락
```

```
impl LockedFile {
pub fn open_exclusive<P: AsRef<Path>>(p: P) -> io::Result<Self> {
let f = OpenOptions::new()
.read(true)
.write(true)
.create_new(true)
.open(p)?;
f.lock_exclusive()?; // 배타적 파일 락
Ok(LockedFile(f))
}
}
```

## 캐시 계층 구조

1. **L1 캐시: 메모리 내 캐시**
   - 가장 자주 사용되는 데이터
   - 원자적 연산을 통한 빠른 접근
2. **L2 캐시: 메모리 맵핑**
   - 디스크 파일을 메모리에 맵핑
   - OS 페이지 캐시 활용
3. **L3 캐시: 디스크 저장소**
   - 영구 저장된 캐시 파일
   - 재시작 후에도 캐시 유지

# 성능 모니터링 및 튜닝

## 성능 측정 지표

1. **처리량 메트릭**

- 초당 처리되는 노드 수
- 메모리 대역폭 사용률
- GPU 사용률

2. **지연 시간 메트릭**

- 캐시 미스 비율
- 디스크 I/O 대기 시간
- 스레드 동기화 오버헤드

## 튜닝 가이드라인

1. **하드웨어별 최적화**

```
# GPU 가속 활성화
export FIL_PROOFS_USE_GPU_TREE_BUILDER=1
export FIL_PROOFS_USE_GPU_COLUMN_BUILDER=1
# 멀티코어 SDR 활성화
export FIL_PROOFS_USE_MULTICORE_SDR=1
export FIL_PROOFS_MULTICORE_SDR_PRODUCERS=4
```

2. **메모리 사용량 조절**

```
# 캐시 크기 조절
```

```
export FIL_PROOFS_SDR_PARENTS_CACHE_SIZE=4096

export FIL_PROOFS_MAX_GPU_TREE_BATCH_SIZE=500000
```

3. **디스크 I/O 최적화**

```
# 배치 크기 조절

export FIL_PROOFS_COLUMN_WRITE_BATCH_SIZE=524288
```

## 결론

파일코인 proof 시스템의 성능 최적화는 다층적 접근을 통해 이루어집니다:

- **GPU 가속**: Poseidon 해시 연산의 대폭적인 성능 향상
- **멀티코어 처리**: 하드웨어 토폴로지를 고려한 최적 스레드 배치
- **메모리 최적화**: 부분 캐시 로딩과 더블 버퍼링을 통한 메모리 효율성
- **디스크 I/O**: 메모리 맵핑과 배치 처리를 통한 I/O 성능 향상

이러한 최적화 기법들이 결합되어 대용량 섹터 처리에서도 실용적인 성능을 달성할 수 있습니다.